

Verificación de código y suite de pruebas

DigitalBank Backend

Auditoría línea por línea de HU-01, HU-05, HU-06, HU-10 y HU-18, pruebas unitarias añadidas, ejecución de la suite completa y hallazgos para el registro de incidencias.

Repositorio	DigitalBank-Backend / digital-bank
Stack	Spring Boot 4.1.1 · Java 21 · PostgreSQL · Flyway · JWT
Fecha	19 de septiembre de 2026
Preparado por	Claude Code (verificación asistida) — revisar y validar antes de usar en producción
Alcance	Solo lectura de código + creación de pruebas y configuración local. Sin acciones de git.

Tabla de contenido

1. Resumen ejecutivo
2. Metodología
3. Verificación HU-01 — Registro de cliente
4. Verificación HU-18 — Login y bloqueo por intentos fallidos
5. Verificación HU-05 — Apertura de cuenta
6. Verificación HU-10 y HU-06 — Depósito y consulta de saldo
7. Pruebas unitarias creadas
8. Bloqueos de entorno encontrados y cómo se resolvieron
9. Ejecución de la suite completa (mvn test)
10. Integración continua (GitHub Actions)
11. SonarQube / JaCoCo
12. Registro de hallazgos para bug tracker
13. Instrucciones para reproducir este entorno
14. Anexo — archivos creados y modificados

1. Resumen ejecutivo

Se verificó contra el código real (no contra documentación ni supuestos) el comportamiento de cinco historias de usuario ya implementadas en [digital-bank](#): registro de cliente (HU-01), login con bloqueo por intentos fallidos (HU-18), apertura de cuenta (HU-05), depósito (HU-10) y consulta de saldo (HU-06). Cada afirmación de este reporte está respaldada por una cita de archivo y línea específica.

25 / 26 tests pasan

3 hallazgos de código (severidad media)

3 hallazgos de entorno / documentación

Qué se hizo

- Se leyeron y verificaron línea por línea 5 controladores, 6 servicios, 4 entidades de dominio, DTOs, excepciones y las migraciones Flyway.
- Se confirmaron o corrigieron 5 supuestos previamente documentados sobre estas historias de usuario (ver secciones 3–6).
- Se crearon **4 archivos de prueba nuevos** con **20 métodos de test**, siguiendo el estilo ya existente en [CuentaServiceImplTest](#).
- Se ejecutó la suite completa (`mvn test`): **26 pruebas, 25 exitosas, 1 falla preexistente no relacionada con este trabajo**.
- Se creó el workflow de CI ([.github/workflows/ci.yml](#)) y se configuró JaCoCo + propiedades de SonarQube en [pom.xml](#).
- No se ejecutó ningún comando de git ni se modificó [CuentaController](#) (por instrucción explícita, pendiente de decisión).

Qué se encontró

- **3 hallazgos de código** con impacto real en el sistema (autorización inconsistente, validación de monto ausente en el dominio, test de contexto no aislado de infraestructura real) — detallados con severidad en la sección 12.
- **2 bloqueos de entorno local** (mirror de Maven institucional inalcanzable fuera de la red UdeA, y JDK activo incorrecto) que no afectan el código pero sí la reproducibilidad del build — documentados con la solución exacta aplicada.
- **1 desalineación de documentación**: el registro de cliente no usa "nombre completo" sino `nombres` + `apellidos` por separado, y exige `telefono` (10 dígitos) — campo no documentado previamente.

2. Metodología

Para cada historia de usuario se siguió el mismo proceso:

1. **Lectura directa del código fuente** (controlador, DTO, servicio, entidad, excepciones) — sin inferir comportamiento a partir de nombres de métodos o de la documentación existente.
2. **Cita exacta** de archivo y número de línea para cada afirmación.
3. **Marcado explícito** cuando algo documentado previamente no coincidía con el código, o cuando una validación no existía, en vez de asumir que sí estaba implementada.
4. **Consulta al equipo** (mediante preguntas puntuales) en los dos únicos puntos donde el comportamiento real del código no permitía escribir la prueba solicitada tal como estaba nombrada, para no fabricar un test engañoso.

Convención usada en este documento

Cada hallazgo confirmado contra el código se marca con . Cada hallazgo que representa un problema real se marca con  y aparece también consolidado en la sección 12 para pasar directamente al bug tracker.

3. Verificación HU-01 — Registro de cliente

Archivos verificados: `cliente/controller/ClienteController.java`, `cliente/dto/ClienteRegistroRequest.java`, `cliente/service/ClienteServiceImpl.java`.

a. Campos exigidos por el DTO

✓ Confirmado (con corrección)

El registro **no** pide un campo único "nombre completo": exige `nombres` y `apellidos` por separado. Además existe un campo **no documentado previamente**: `telefono`, obligatorio, con patrón exacto de 10 dígitos.

Campo	Regla de validación
<code>tipoDocumento</code>	<code>@NotNull</code>
<code>numeroDocumento</code>	<code>@NotBlank</code>
<code>nombres</code>	<code>@NotBlank</code>
<code>apellidos</code>	<code>@NotBlank</code>
<code>fechaNacimiento</code>	<code>@NotNull @Past</code>
<code>telefono</code>	<code>@NotBlank @Pattern(\d{10})</code>
<code>email</code>	<code>@NotBlank @Email</code>
<code>password</code>	<code>@NotBlank @Size(min = 8)</code>

b. Documento duplicado

✓ Confirmado

`ClienteServiceImpl.java:58-62` — `clienteRepository.existsByTipoDocumentoAndNumeroDocumento(...)`, respaldado por el constraint `uk_cliente_documento` en base de datos.

c. Correo duplicado

✓ Confirmado

`ClienteServiceImpl.java:64-68` — `clienteRepository.existsByEmail(...)`, respaldado por `uk_cliente_email`.

d. Mayoría de edad

✓ Confirmado

`ClienteServiceImpl.java:70-75`:

```
int edad = Period.between(fechaNacimiento, LocalDate.now()).getYears();
if (edad < EDAD_MINIMA) { throw new ClienteMenorDeEdadException(...); }
// EDAD_MINIMA = 18 (constante, línea 21)
```

e. Códigos de respuesta HTTP

Caso	Código	Excepción
Éxito	201 CREATED	—
Documento duplicado	409 CONFLICT	ClienteYaExisteException
Correo duplicado	409 CONFLICT	ClienteYaExisteException (mismo mensaje, no distingue campo)
Menor de edad	400 BAD_REQUEST	ClienteMenorDeEdadException

4. Verificación HU-18 — Login y bloqueo por intentos fallidos

Archivos verificados: `auth/controller/AuthController.java`, `auth/service/AuthServiceImpl.java`, `auth/entity/Usuario.java`.

a. Intentos que activan el bloqueo

✓ Confirmado: 3

`Usuario.java:11` — `MAX_INTENTOS_FALLIDOS = 3`

b. Duración del bloqueo

✓ Confirmado: 1 hora

`Usuario.java:12` — `HORAS_BLOQUEO = 1`

c. ¿Desbloqueo automático o manual?

✓ Confirmado: automático

No requiere intervención administrativa. `AuthServiceImpl.java:37-39`:

```
if (usuario.bloqueoExpiro()) {  
    usuario.desbloquear();  
}  
// se evalúa en cada intento de login, antes de revisar la contraseña
```

d. Reinicio del conteo tras login exitoso

✓ Confirmado

`Usuario.java:82-87`, método `registrarLoginExitoso()`, línea exacta: `this.intentosFallidos = 0;`

e. Mensaje de error de credenciales

✓ Confirmado: genérico

Tanto correo inexistente como contraseña incorrecta lanzan la misma excepción

`CredencialesInvalidasException` con el mismo texto "*Credenciales inválidas*" y `401 UNAUTHORIZED` — diseño correcto anti-enumeración de usuarios.

5. Verificación HU-05 — Apertura de cuenta

Archivos verificados: `cuenta/controller/CuentaController.java`, `cuenta/service/CuentaServiceImpl.java`.

a. ¿Valida que el cliente esté ACTIVO?

✓ Confirmado — sí existe

`CuentaServiceImpl.java:49-51`:

```
if (cliente.getEstado() != EstadoCliente.ACTIVO) {  
    throw new ClienteNoHabilitadoException("Cliente no habilitado");  
}
```

b. Tipos de cuenta

✓ Confirmado: solo 2

`TipoCuenta.java` — AHORROS, CORRIENTE.

c. Monto de apertura negativo

✓ Confirmado — sí hay validación de aplicación

`AperturaCuentaRequest.java:22` — `@PositiveOrZero`, se rechaza antes de llegar al servicio (400 BAD_REQUEST). El constraint `ck_cuenta_saldo_no_negativo` de BD es solo una segunda capa de defensa.

d. Comparación de extracción de identidad

Hallazgo

Confirmado: son dos mecanismos distintos e inconsistentes dentro del mismo módulo.

Controlador	Mecanismo
<code>SaldoController.java:26-30</code>	<code>UUID usuarioId = (UUID) authentication.getPrincipal();</code>
<code>CuentaController.java:36-58</code>	<code>jwtService.extraerClienteId(header.substring(7))</code> – decodifica el JWT manualmente

e. ¿Tiene @PreAuthorize?

Hallazgo

No tiene. `CuentaController.java:35-43`, método `abrirCuenta`, no tiene ninguna anotación de rol — en contraste, el método `depositar` del mismo archivo sí tiene `@PreAuthorize("hasAnyRole('ADMIN','CAJERO')")` (línea 45). Cualquier rol autenticado puede llegar a este endpoint sin restricción explícita.

Decisión pendiente (Tarea 9 — no se corrigió)

Por instrucción explícita, `CuentaController.abrirCuenta` no fue modificado en esta sesión. Este hallazgo queda documentado para que el equipo decida la corrección en una sesión aparte, dado que impacta lógica de producción.

6. Verificación HU-10 y HU-06 — Depósito y consulta de saldo

Archivos verificados: `cuenta/service/DepositoServiceImpl.java`, `cuenta/service/SaldoServiceImpl.java`.

a-b. Monto de depósito en \$0 o negativo

Hallazgo

Ambos casos producen el **mismo mensaje**, "*Monto inválido*", pero la validación **no vive en** `DepositoServiceImpl`: es la anotación `@DecimalMin(value = "0.0", inclusive = false)` del DTO `DepositoRequest`, activada solo por `@Valid` en el controlador. El servicio, si se invoca directamente, no rechaza nada.

c. Cuenta inexistente vs. cuenta de otro cliente

✓ Confirmado — mismo mensaje, por diseño

`SaldoServiceImpl.java:35-38`: para un solicitante con rol CLIENTE, ambos casos lanzan `AccesoNoAutorizadoException` ("Acceso no autorizado", 403) — protección anti-enumeración intencional. Para roles ADMIN/CAJERO sí se distingue (`CuentaNoEncontradaException`, 404), porque para personal autorizado no aplica esa protección.

d. Cálculo del saldo disponible

✓ Confirmado

`SaldoServiceImpl.java:44` — `saldoDisponible = saldoContable.subtract(retencion)`, sin excepciones en el código actual. `retencion` siempre es 0 hoy porque ninguna funcionalidad implementada aún la modifica — el campo existe para uso futuro.

7. Pruebas unitarias creadas

Se crearon 4 archivos nuevos siguiendo el estilo de `CuentaServiceImplTest` (JUnit 5 + Mockito, patrón AAA, `@ExtendWith(MockitoExtension.class)`, `@Mock` / `@InjectMocks`).

Archivo	Tests	Resultado
<code>cliente/service/CuentaServiceImplTest.java</code>	4	4/4
<code>auth/service/AuthServiceImplTest.java</code>	7	7/7
<code>cuenta/service/DepositoServiceImplTest.java</code>	4	4/4
<code>cuenta/service/SaldoServiceImplTest.java</code>	5	5/5

Decisiones consultadas con el equipo durante la construcción

- **Bloqueo expirado:** `Usuario` no expone setter para `fechaBloqueo` ni permite inyectar reloj. Se usó *reflection* para fijar esa fecha en el pasado sobre una instancia real, la única forma de ejercitar `bloqueoExpiro()` / `estaBloqueado()` reales sin tocar código de producción.
- **Monto cero/negativo en depósito:** como `DepositoServiceImpl` no valida el monto (ver sección 6a-b), los tests `registrarDeposito_montoCero_debeRechazar` y `...montoNegativo_debeRechazar` se reescribieron para validar el DTO `DepositoRequest` directamente con `jakarta.validation.Validator` — la capa donde la regla realmente vive — dejando un comentario explícito en el código.

8. Bloqueos de entorno encontrados y cómo se resolvieron

Antes de poder ejecutar cualquier prueba, se presentaron dos bloqueos **de entorno local, no del código del proyecto**.

8.1 Mirror institucional de Maven inalcanzable

`C:\Users\jonat\.m2\settings.xml` (global, fuera del repositorio) fuerza todas las descargas de dependencias a través de un Nexus de la UdeA en una IP privada, inalcanzable desde esta red.

```
bash - mvnw test (primer intento)

[INFO] Scanning for projects ...
[ERROR] [ERROR] Some problems were encountered while processing the POMs:
[FATAL] Non-resolvable parent POM for com.fabricaescuela:digital-bank:0.0.1-SNAPSHOT:
The following artifacts could not be resolved:
    org.springframework.boot:spring-boot-starter-parent:pom:4.1.1 (present, but unavailable):
    Could not transfer artifact ... from/to co.edu.udea.nexus
    (http://172.19.1.106:8081/repository/udea-produccion/):
    Connect to 172.19.1.106:8081 [/172.19.1.106] failed: connect timed out
```

Captura real de la salida de `./mvnw test` ejecutado durante esta sesión.

8.2 JDK activo incorrecto

`JAVA_HOME` apuntaba a JDK 8, incompatible con el proyecto (Java 21, usa `record` y `text blocks`). El compilador de JDK 8 interpreta `record` como un identificador cualquiera, generando errores de sintaxis en cascada en archivos que en realidad están bien escritos.

```
bash - java -version

java version "1.8.0_121"
Java(TM) SE Runtime Environment (build 1.8.0_121-b13)
file.encoding = Cp1252
JAVA_HOME=C:\Program Files\Java\jdk1.8.0_121
```

```
bash - mvnw test (con JDK 8 activo)

[ERROR] COMPILATION ERROR :
[ERROR] ... /auth/dto/LoginRequest.java:[6,8] class, interface, or enum expected
[ERROR] ... /core/config/SecurityConfig.java:[43,54] unclosed string literal
// record y text blocks no existen en Java 8 → el parser se desincroniza
[INFO] 43 errors
```

Solución aplicada (sin tocar configuración global ni archivos del proyecto)

1. Se creó un `settings.xml` temporal en el scratchpad de la sesión, apuntando a Maven Central en vez del Nexus institucional, usado solo con la bandera `-s` en cada comando.
2. Se localizó un JDK 21 ya instalado en la máquina (`C:\Program Files\Eclipse Adoptium\jdk-21.0.10.7-hotspot`) y se sobrescribió `JAVA_HOME` solo para el comando puntual, no de forma persistente.

bash – comando usado para compilar correctamente

```
JAVA_HOME="/c/Program Files/Eclipse Adoptium/jdk-21.0.10.7-hotspot" \  
./mvnw -s "<ruta-al-settings-temporal>/maven-settings-central.xml" test
```

9. Ejecución de la suite completa (mvn test)

```
bash - mvnw test (resultado final)

[INFO] Results:
[ERROR] Errors:
[ERROR]   DigitalBankApplicationTests.contextLoads » IllegalStateException
    Failed to load ApplicationContext for [WebMergedContextConfiguration ...]

[ERROR] Tests run: 26, Failures: 0, Errors: 1, Skipped: 0

[INFO] _____
[INFO] BUILD FAILURE
[INFO] _____
```

26 pruebas ejecutadas · 25 exitosas · 1 error preexistente, no causado por este trabajo.

Desglose por clase (target/surefire-reports)

Clase	Tests	Resultado
DigitalBankApplicationTests	1	0/1 — preexistente
AuthServiceImplTest	7	7/7
ClienteServiceImplTest	4	4/4
CuentaServiceImplTest	5	5/5
DepositoServiceImplTest	4	4/4
SaldoServiceImplTest	5	5/5

Stack trace completo de la única falla

```
bash - surefire-reports / DigitalBankApplicationTests.txt

java.lang.IllegalStateException: Failed to load ApplicationContext ...
Caused by: org.springframework.beans.factory.BeanCreationException:
  Error creating bean with name 'entityManagerFactory' ...
  Failed to initialize dependency 'flywayInitializer' ...
Caused by: org.flywaydb.core.internal.exception.sqlExceptions.FlywaySqlUnableToConnectToDbException:
  Unable to obtain connection from database: El intento de conexión falló.
  SQL State : 08001
Caused by: org.postgresql.util.PSQLException: El intento de conexión falló.
  at
org.postgresql.core.v3.ConnectionFactoryImpl.openConnectionImpl(ConnectionFactoryImpl.java:439)
  at org.postgresql.core.ConnectionFactory.openConnection(ConnectionFactory.java:52)
  at org.postgresql.jdbc.PgConnection.<init>(PgConnection.java:295)
  at org.postgresql.Driver.makeConnection(Driver.java:437)
  at org.postgresql.Driver.connect(Driver.java:305)
  at com.zaxxer.hikari.pool.HikariPool.createPoolEntry(HikariPool.java:488)
  at com.zaxxer.hikari.pool.HikariPool.checkFailFast(HikariPool.java:576)
  at com.zaxxer.hikari.HikariDataSource.getConnection(HikariDataSource.java:111)
Caused by: java.net.UnknownHostException: ${DB_HOST}
  at java.base/sun.nio.ch.NioSocketImpl.connect(NioSocketImpl.java:567)
  at java.base/java.net.Socket.connect(Socket.java:751)
  at org.postgresql.core.PGStream.createSocket(PGStream.java:270)
  at org.postgresql.core.v3.ConnectionFactoryImpl.tryConnect(ConnectionFactoryImpl.java:221)
```

Causa raíz

UnknownHostException: \${DB_HOST} — Spring intenta resolver literalmente el nombre `${DB_HOST}` como host DNS porque esa variable de entorno no está definida en esta máquina.

`DigitalBankApplicationTests` es un `@SpringBootTest` que arranca el contexto completo y necesita una base de datos real — no tiene perfil de test aislado. Ver hallazgo #3 en la sección 12.

10. Integración continua (GitHub Actions)

Se creó `.github/workflows/ci.yml` con el contenido acordado. No se subió a ningún repositorio remoto.

```
.github/workflows/ci.yml

name: CI

on:
  pull_request:
    branches: [ main ]
  push:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK 21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
      - name: Run tests
        run: ./mvnw test
```

Advertencia para antes de activarlo

Este workflow fallará en GitHub Actions por el mismo motivo de la sección 9: no hay

`DB_HOST` / `JWT_SECRET` configurados. Antes de habilitarlo en serio, decidir entre: (a) agregar un servicio Postgres al job con secrets del repo, o (b) excluir `DigitalBankApplicationTests` de la ejecución en CI hasta que tenga un perfil de test aislado.

11. SonarQube / JaCoCo

Se agregaron a `pom.xml` las propiedades de SonarQube y el plugin `jacoco-maven-plugin` 0.8.12 (goals `prepare-agent` y `report`, este último enlazado a la fase `test`).

```
bash - mvn test tras agregar JaCoCo

find target -path "*jacoco*"
target/jacoco.exec
// jacoco.xml NO se generó: el build sigue fallando por contextLoads (sección 9),
// y Maven aborta la fase "test" antes de llegar al goal jacoco:report
```

Para confirmar que **la configuración del plugin en sí es correcta**, se repitió el comando con `-Dmaven.test.failure.ignore=true` (solo una bandera de línea de comandos, ningún archivo modificado):

```
bash - mvn test -Dmaven.test.failure.ignore=true

[INFO] --- jacoco:0.8.12:report (report) @ digital-bank ---
[INFO] Loading execution data file .../target/jacoco.exec
[INFO] Analyzed bundle 'digital-bank' with 52 classes
[INFO] _____
[INFO] BUILD SUCCESS
[INFO] _____
```

Con esta bandera sí se generó `target/site/jacoco/jacoco.xml` completo, con reporte HTML por clase (52 clases analizadas).

Conclusión

La configuración de JaCoCo y Sonar en `pom.xml` es correcta y funcional. El bloqueo para generar el XML en un `mvn test` normal es exclusivamente el mismo test de contexto sin aislar (sección 9) — resolver ese hallazgo destraba automáticamente tanto CI como SonarCloud.

12. Registro de hallazgos — listo para bug tracker

Cada fila puede convertirse directamente en un ticket. Ninguno de estos hallazgos fue corregido en esta sesión (excepto los tests añadidos), por instrucción explícita de mantener el trabajo de código separado del de verificación.

#1 — Autorización inconsistente en apertura de cuenta

Severidad: Media

Componente	<code>cuenta/controller/CuentaController.java:35-58</code>
Descripción	El endpoint <code>POST /api/cuentas</code> no tiene <code>@PreAuthorize</code> y extrae el <code>clienteld</code> decodificando el header <code>Authorization</code> manualmente, duplicando lógica que el filtro JWT ya resuelve vía <code>Authentication</code> (como sí hace <code>SaldoController</code>).
Riesgo	Cualquier rol autenticado (incluido ADMIN/CAJERO sin <code>clienteld</code> propio) puede llegar al método; el comportamiento correcto actual depende de que <code>clienteldAutenticado</code> termine siendo <code>null</code> y falle más abajo, no de un control de acceso explícito.
Sugerencia	Usar <code>Authentication</code> como en <code>SaldoController</code> y agregar <code>@PreAuthorize("hasRole('CLIENTE')")</code> explícito.

#2 — Validación de monto ausente en el dominio de depósitos

Severidad: Media

Componente	<code>cuenta/service/DepositoServiceImpl.java</code>
Descripción	La regla "monto > 0" vive únicamente en <code>@DecimalMin</code> del DTO <code>DepositoRequest</code> , activada solo por <code>@Valid</code> en el controlador. El servicio de dominio no la revalida.
Riesgo	Cualquier otro punto de entrada futuro que invoque <code>DepositoServiceImpl</code> directamente (job interno, otro controlador, mensaje de cola) podría acreditar montos en cero o negativos sin control.
Sugerencia	Repetir la validación (o delegarla a un método de <code>Cuenta.acreditar()</code>) dentro del dominio, no solo en el DTO.

#3 — Test de contexto no aislado de infraestructura real

Severidad: Media

Componente	<code>src/test/java/.../DigitalBankApplicationTests.java</code>
Descripción	<code>@SpringBootTest</code> que arranca el contexto completo (incluida migración Flyway) contra la base de datos configurada en <code>application.properties</code> , sin perfil de test ni valores por defecto para <code>DB_HOST</code> .
Riesgo	Bloquea <code>mvn test</code> en cualquier máquina/CI sin esas variables de entorno configuradas — incluido el workflow de CI recién creado (sección 10).

Sugerencia	Agregar un perfil <code>application-test.properties</code> con H2/Testcontainers, o excluir este test de la ejecución estándar hasta tenerlo.
-------------------	---

#4 y #5 — Bloqueos de entorno local

Severidad: Baja (no bloquea producción)

#4	<code>C:\Users\jonat\.m2\settings.xml</code> fuerza un mirror institucional (IP privada UdeA) inalcanzable fuera de esa red — cualquier desarrollador remoto no puede compilar sin ajustar su configuración local.
#5	<code>JAVA_HOME</code> del sistema apunta a JDK 8; el proyecto requiere Java 21. Hay un JDK 21 instalado pero no es el activo por defecto.

#6 — Documentación de HU-01 desactualizada

Severidad: Baja

El registro de cliente no usa "nombre completo" sino `nombres` + `apellidos` separados, y exige `telefono` obligatorio (10 dígitos), campo no reflejado en el Gherkin/documentación previa de la historia.

13. Instrucciones para reproducir este entorno

13.1 Requisitos previos

- JDK 21 instalado (verificar con `java -version`). Si el sistema tiene varias versiones, confirmar cuál activa `JAVA_HOME`.
- Acceso a un repositorio Maven que resuelva `spring-boot-starter-parent:4.1.1` (Maven Central, o el Nexus de la UdeA únicamente si se está en su red/VPN).

13.2 Compilar y correr los tests unitarios (sin necesitar base de datos)

```
bash

# Si JAVA_HOME no apunta a un JDK 21, sobreescribirlo solo para el comando:
JAVA_HOME="/ruta/a/jdk-21" ./mvnw test -Dtest='!DigitalBankApplicationTests'
```

Excluir `DigitalBankApplicationTests` evita el único fallo relacionado con infraestructura (hallazgo #3) y permite validar rápido los 25 tests de servicio.

13.3 Correr la suite completa (requiere base de datos real)

Definir antes de ejecutar `mvn test` sin exclusiones:

Variable	Ejemplo
DB_HOST	db.xxxx.supabase.co
DB_PORT	5432 (opcional, es el valor por defecto)
DB_NAME	postgres
DB_USERNAME	postgres
DB_PASSWORD	*****
JWT_SECRET	una cadena larga y aleatoria

13.4 Generar el reporte de cobertura JaCoCo

```
bash

./mvnw test -Dtest='!DigitalBankApplicationTests'
# abrir target/site/jacoco/index.html en el navegador
```

13.5 Antes de activar SonarCloud

Confirmar que `sonar.projectKey` coincide con el proyecto creado en SonarCloud, y ejecutar con el token del organización:

bash

```
./mvnw test -Dtest='!DigitalBankApplicationTests' sonar:sonar \  
-Dsonar.token=$SONAR_TOKEN
```

14. Anexo — archivos creados y modificados

Ningún comando de git fue ejecutado en ningún momento de esta sesión.

Archivos creados

- `src/test/java/com/fabricaescuela/digitalbank/cliente/service/ClienteServiceImplTest.java`
- `src/test/java/com/fabricaescuela/digitalbank/auth/service/AuthServiceImplTest.java`
- `src/test/java/com/fabricaescuela/digitalbank/cuenta/service/DepositoServiceImplTest.java`
- `src/test/java/com/fabricaescuela/digitalbank/cuenta/service/SaldoServiceImplTest.java`
- `.github/workflows/ci.yml`

Archivos modificados

- `pom.xml`
 - propiedades `sonar.*` + plugin `jacoco-maven-plugin`

Reporte generado a partir de una sesión de verificación asistida por Claude Code el 19 de septiembre de 2026. Todas las citas de código corresponden al estado del repositorio en ese momento; revalidar antes de crear tickets si el código ha cambiado desde entonces.